

JSS Mahavidyapeetha
JSS SCIENCE AND TECHNOLOGY UNIVERSITY
JSS Technical Institutions' Campus, Mysuru – 570006



Internship/Industrial Training
24SDS310P

MOVIE RECOMMENDATION SYSTEMS"

A Report submitted in partial fulfillment of curriculum prescribed for the
award of the degree of

MASTER OF TECHNOLOGY
IN
DATA SCIENCE

by

CHANDAN A 01JST24PDS003

Institute Name

CAD DESK , MYSURU

Duration of Internship

(24/11/2025 – 17/01/2026)

DEPARTMENT OF INFORMATION SCIENCE AND ENGINEERING
JSS SCIENCE AND TECHNOLOGY UNIVERSITY , MYSURU - 570006

November 2025

ABSTRACT

In the era of digital entertainment, the rapid growth of online streaming platforms has resulted in an overwhelming amount of multimedia content, making it difficult for users to identify movies that match their preferences. A Movie Recommendation System plays a crucial role in addressing this challenge by providing personalized suggestions based on user interests and behaviour. This project focuses on the design and implementation of an intelligent Movie Recommendation System using machine learning techniques.

The proposed system employs a hybrid recommendation approach that combines content-based filtering and collaborative filtering methods. Content-based filtering analyzes movie attributes such as genres, cast, director, and plot descriptions using TF-IDF vectorization and cosine similarity to recommend movies similar to those previously liked by the user. Collaborative filtering leverages user-item interaction data and rating patterns to identify similarities between users or movies, enabling recommendations based on collective user behaviour. By integrating both techniques, the system overcomes limitations such as the cold-start problem and improves recommendation accuracy.

The system is implemented using Python, with libraries such as Pandas, NumPy, and Scikit-learn for data preprocessing and model development. A RESTful API is developed using FastAPI to expose recommendation services, making the system scalable and suitable for real-world applications. Experimental results demonstrate that the hybrid approach provides more relevant and personalized movie recommendations compared to individual filtering techniques.

CONTENTS

Abstract	i
	Page No.
CHAPTER 1: INSTITUTE PROFILE	1
CHAPTER 2 : INTRODUCTION	3
CHAPTER 3 : PROJECT WORK	
3.1 Work Assigned/Project Title	4
3.2 Objectives and Problem Statement	5
3.3 Methodology/Workflow	6 – 11
3.4 System Architecture	12 – 13
3.5 Implementation details	14
3.6 Evaluation and Analysis	15
3.7 Results and Discussions	16 - 18
3.8 Strengths, Limitations, and Future Work	19 - 20
CHAPTER 4 : CONCLUSION	21
CHAPTER 5 : APPLICATIONS	22
CHAPTER 6 : REFERENCES	23

Chapter 1

ABOUT THE INSTITUTE

CAD DESK provides professional software training on a vast array of courses in the fields of CAD, CAM and CAE. Since its founding in 2007, CAD DESK has provided training to over 1.5 Lakhs+ students and professionals of various countries.

Since its inception, in a very short span of time itself, with the qualitative services of training and well-designed training modules, CAD DESK has expanded its network and become the topmost leading franchise network. CAD DESK proudly owns 170+ franchise centers in various parts of the country. Furthermore, we are now expanding our network in the various countries in the world.

CAD DESK is the Authorized Training Partner of Autodesk, PTC, Bentley Systems and Dassault Systems. It enables us to work on a wide range of software – Civil CAD, Mechanical CAD, Electrical CAD.

Quality education being the motto, CAD DESK focus on the overall development of every student and that's why our highly trained faculties not just deliver the theoretical contents but put a great emphasis on quality and practical knowledge of each software courses, which helps students to get immediate placement after the course completion. Each software course is professionally designed in such a way that it covers all the topics and tools and bridges the gap between students and industry needs.

CAD DESK provides various well documented and easy to understand course material to all the students. For the effective learning, students are also encouraged to work on Live projects and industrial visits are planned during the training period. At the end of the course, Certification process is initiated once the feedback and project are submitted.

CAD DESK certificates are completely secure and authorized. To check the validity of the certificate, students can scan the QR code on their certificate. Also, to check the authenticity of the certificate online, students can enter their Registration number in the Certificates option on official website of CAD DESK India.

With the need of the hour, CAD DESK along with IT DESK has incorporated complete Online Software Training. It has facilitated the students to take Online Training on Mechanical, Civil/Architecture, Electrical, CS/IT and Creative arts as per their requirement while sitting at their own place. All the trainings in this system are Live Trainings, which means students can interact with the faculties and get their queries answered. With this system, we are trying to accumulate all the trainings such as academic training, internship program and all short-term skill development.

VISION

CAD DESK offers comprehensive software training on CAD, CAM, CAE and CS/IT. Our training programs do not just focus on theoretical concepts, but its practical aspects fulfill the career objectives of the students as well as professionals.

MISSION

CAD DESK offers comprehensive software training on CAD, CAM, CAE and CS/IT. Our training programs do not just focus on theoretical concepts, but its practical aspects fulfill the career objectives of the students as well as professionals.

Address: 8th Main Road, 5th Cross Road, near Javaregowda Park, Saraswathipuram, Mysuru, Karnataka 570009.

For more details, visit <https://www.caddeskmysuru.com>

Chapter - 2

INTRODUCTION

Personalized recommendation systems have become a critical component of modern digital platforms such as streaming services, e-commerce sites, and social media applications. In the movie domain, recommender systems help users efficiently discover relevant content from large catalogs, improving user satisfaction and engagement while increasing platform watch-time and retention. This project develops a movie recommendation system that integrates content-based filtering, collaborative filtering, and hybrid recommendation strategies using synthetic but realistic datasets of movies, users, and ratings.

Movie Recommendation System is a software application that suggests relevant movies to users by analyzing their interests, preferences, and past interactions. These systems aim to enhance user experience by delivering personalized recommendations, increasing user engagement, and improving customer satisfaction. Recommendation systems are a key application of machine learning, data mining, and information retrieval techniques.

The rapid advancement of digital technology and the widespread adoption of online streaming platforms have transformed the way users consume multimedia content. Platforms such as Netflix, Amazon Prime, and Disney+ offer thousands of movies and shows, creating an issue of information overload where users find it difficult to choose content that aligns with their preferences. To address this challenge, intelligent recommendation systems have become an essential component of modern digital platforms.

Chapter - 3

PROJECT WORK

3.1 Work Assigned :

The work assigned for this project involves the design, development, and evaluation of a Movie Recommendation System using machine learning techniques. The major tasks assigned are as follows:

- i. Studying the fundamentals of recommendation systems and understanding content-based, collaborative, and hybrid filtering techniques.
- ii. Collecting and preprocessing movie metadata and user rating datasets for effective model training.
- iii. Designing a content-based recommendation module using textual features such as genre, cast, director, and movie descriptions.
- iv. Implementing collaborative filtering techniques based on user–item interaction and rating patterns.
- v. Integrating both recommendation approaches to develop a hybrid recommendation system that improves accuracy and reduces limitations such as the cold-start problem.
- vi. Developing the system using Python and machine learning libraries such as Pandas, NumPy, and Scikit-learn.
- vii. Creating a RESTful API using FastAPI to expose recommendation functionalities in a scalable manner.
- viii. Evaluating the performance of the recommendation system based on relevance and recommendation quality.

3.2 Objectives and Problem Statement :

To design and implement a modular recommendation engine that supports content-based, user-based collaborative, and item-based collaborative filtering.

To construct and analyze similarity matrices for users and items, enabling efficient neighborhood-based recommendation.

To integrate a hybrid mechanism that combines content and collaborative scores, improving robustness and overall recommendation quality.

The core problem addressed is how to recommend a ranked list of movies to each user that reflects both individual preferences and collective behavior patterns, under realistic constraints of data sparsity and cold-start scenarios. Traditional single-method recommenders either rely solely on item attributes (content-based) or solely on interaction data (collaborative), and each has limitations in terms of exploration, diversity, and handling new items or users.

3.3 Methodology/Workflow :

i) Data preprocessing and feature engineering:

The first step involves reading the CSV files into Pandas dataframes and constructing derived representations suitable for different recommendation paradigms. For content-based filtering, textual features from genres, director, and actors are concatenated into a single descriptive string for each movie, which is then vectorized using TF-IDF to capture term importance across the catalog. For collaborative filtering, the ratings.csv records are pivoted into a user–item rating matrix, where rows represent users, columns represent movies, and cells hold observed ratings or remain missing for unrated items.

Basic cleaning includes normalizing text (e.g., lower-casing, stripping whitespace) and validating rating ranges, although the synthetic data is already consistent by design. The resulting matrices are typically stored as sparse matrices to optimize memory usage and computation time, especially for similarity calculations using SciPy and scikit-learn utilities.

ii) Content-based filtering :

Content-based recommendations are generated by analyzing item attributes only, independent of other users' behavior. The steps are:

- Build a TF-IDF matrix where each movie's concatenated content string is represented as a high-dimensional vector.
- Compute pairwise cosine similarity between movie vectors, resulting in a content similarity matrix of size 50×50 .
- For a given query movie, sort other movies by descending similarity score and return the top-N as recommendations.
- This approach captures semantic proximity between movies, for example grouping together titles that share similar genres (e.g., "Fantasy" and "Sci-Fi") or recurring collaborations between particular directors and actors. The cosine similarity scores lie in the range $[0,1]$, with 1 on the diagonal, and are used directly as ranking scores.

iii) User-based collaborative filtering:

User-based collaborative filtering exploits the idea that users with similar historical rating patterns are likely to appreciate similar items. The process includes:

- Construct a user–movie rating matrix from ratings.csv with missing values for unknown ratings.
- Compute user–user similarity (e.g., via cosine similarity on centered rating vectors), producing a 100×100 user similarity matrix where values typically range from -1 to 1 .
- For a target user, identify the top-K most similar users (neighbors).
- Predict ratings for unseen movies using a weighted average of neighbors' ratings, where weights are given by user–user similarities.
- The algorithm then recommends movies with the highest predicted ratings that the user has not rated yet. This method is particularly effective at discovering “serendipitous” items that share an audience with the user’s preferences even if they are content-wise diverse.

iv) Item-based collaborative filtering :

Item-based collaborative filtering flips the perspective and focuses on similarities between movies derived from user rating patterns. The steps are:

- From the user–movie rating matrix, compute item–item similarity (e.g., cosine similarity on columns), generating a 50×50 item similarity matrix.
- For each candidate movie, estimate the expected rating by a user using a similarity-weighted average of the user's ratings for similar movies.
- Rank candidate movies by predicted rating and return the top-N.
- Item-based approaches are typically more stable than user-based ones when item characteristics and rating patterns are stationary over time, and they allow efficient precomputation of the similarity matrix for online use.

v) Hybrid recommendation strategy :

The hybrid recommender aims to combine the strengths of content-based and collaborative methods while mitigating their weaknesses. The implemented hybrid strategy operates as follows:

- Compute a collaborative prediction score for each candidate movie using either user-based or item-based collaborative filtering.
- Compute a content similarity score for each movie relative to the user's historically liked or highly rated movies, focusing especially on matching the user's favorite genres from users.csv.

Combine the two scores using a weighted linear combination:

CollaborativeScore

HybridScore= α ·ContentScore+(1- α)·CollaborativeScore

where ,

α = content_weight

1- α = collab_weight

By tuning α , the system can emphasize either content similarity (beneficial in cold-start scenarios for new movies) or collaborative signals (beneficial when ample rating data exist). This hybrid approach provides more robust and diverse recommendations and is closer to strategies adopted in commercial platforms.

vi) Dataset Description :

a) Movie dataset

The movies.csv file contains a curated catalog of 50 fictional Hollywood-style movies with rich metadata suitable for content-based recommendation. Each record includes:

movie_id: Unique numeric identifier

title: Movie title

year: Release year between 1990 and 2024

genres: Pipe-separated list of genres (e.g., “Comedy”, “Action”, “Thriller”)

director: Single director name

actors: Multiple high-profile actors concatenated in one field

runtime: Approximate duration in minutes

avg_rating: Aggregate mean rating on a 1–10 scale

budget_millions: Approximate budget in millions of dollars

The catalog spans a wide variety of genres and combinations (e.g., “Fantasy”, “Sci-Fi”, “Horror”, “Documentary”, “War”) and deliberately mixes directors and actors across different genre patterns to avoid trivial feature correlations and to stress-test the content-based model. The movie attributes are explicitly designed to support text feature engineering (genres, director, actors) and numeric analysis (runtime, average rating, budget).

b) User dataset

The users.csv file represents 100 synthetic users, including demographic and preference-oriented attributes. For each user, the following fields are provided:

user_id: Unique numeric identifier

age: Age spanning approximately 18–70 years

gender: {M, F, Other}

favorite_genres: Pipe-separated list of preferred genres, such as “Action”, “Romance”, “Documentary”, “Sci-Fi”

join_date: Date when the user joined the platform between 2023 and 2025

The favorite genre combinations are heterogeneous (e.g. “Action|Biography|Mystery|Documentary”, “Romance|Mystery|Horror”, “Sci-Fi|Comedy”), creating a realistic diversity in user tastes that can be exploited by hybrid and content-aware logic.

The join dates are distributed over multiple years, which allows potential temporal analysis and can represent evolving user populations and cold-start scenarios for newer users.

c) Ratings dataset

The ratings.csv file consists of approximately 1,000 user–movie interaction records, representing explicit ratings between 1.5 and 5.0, with associated timestamps from late 2024 to 2025. Each row contains:

user_id

movie_id

rating: Floating-point rating

timestamp: Rating date

Given that there are 100 users and 50 movies, the theoretical user–item matrix has 5,000 possible entries, but only around 1,000 are observed, yielding roughly 80% sparsity, which is typical in real-world recommendation settings. The ratings per user and per movie are roughly balanced (around 10 ratings per user and 20 per movie on average), which provides sufficient density for neighborhood-based collaborative methods while still reflecting realistic sparsity constraints.

d) Summary statistics

Key dataset characteristics are:

Total movies: 50

Total users: 100

Total ratings: ~1000

Matrix sparsity: ~80%

Rating range: 1.0–5.0

Average ratings/user: ≈ 10 ; average ratings/movie: ≈ 20 .

These properties make the dataset small enough for rapid experimentation and visualization, yet complex enough to demonstrate full end-to-end recommendation workflows.

3.4 System Architecture:

The project is structured as a modular Python-based recommendation system designed to be easily integrated into a full-stack web application. The main components are:

- `generate_data.py`: Script to generate and persist synthetic movie, user, and rating datasets as CSV files.
- `recommendation_engine.py`: Core `MovieRecommendationSystem` class implementing content-based, collaborative, and hybrid algorithms.
- `movie_recommendation_demo.ipynb`: Jupyter notebook for interactive experimentation, visualization, and evaluation.

Frontend/backend architecture (as designed in the document) using a Next.js UI, FastAPI backend, and an ML engine module that consumes the CSV datasets for inference.

The architecture follows a typical separation of concerns: data layer (CSV datasets), ML logic layer (recommendation engine), and application layer (API and UI), allowing the engine to be reused and extended for more advanced models or real databases later.

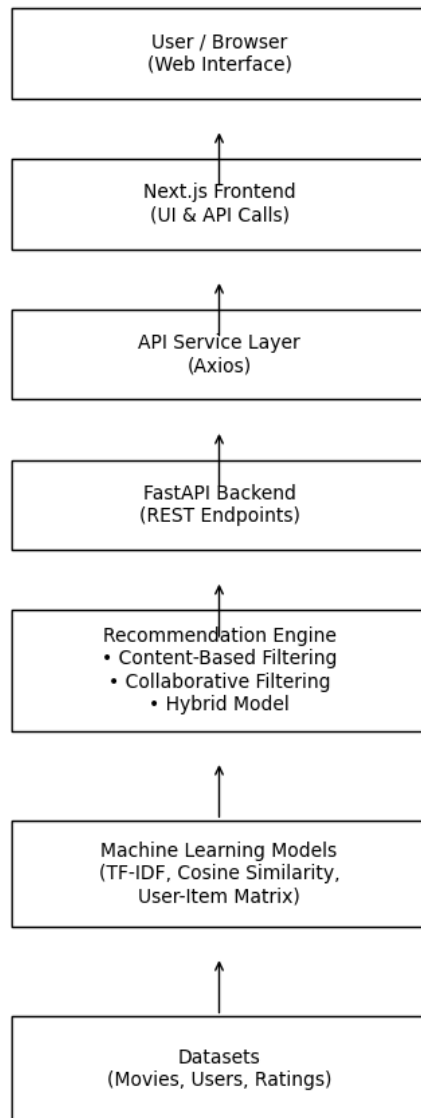


Fig : Flow diagram of Movie Recommendation System

3.5 Implementation

The implementation is done in Python 3.8+ using standard data science and ML libraries. Key dependencies include:

Pandas and NumPy for data loading, cleaning, and matrix manipulation.

Scikit-learn for TF-IDF vectorization and cosine similarity computations.

SciPy for sparse matrix operations to handle the rating and similarity matrices efficiently.

Matplotlib and Seaborn in the notebook for plotting distributions, heatmaps of similarity matrices, and other visualizations.

The main class Movie Recommendation System encapsulates the logic to:

Load and preprocess datasets.

Build content similarity matrices (`build_content_similarity`).

Construct collaborative structures (`build_collaborative_filtering`).

Expose high-level methods such as `content_based_recommendations`, `collaborative_user_based`, `collaborative_item_based`, and `hybrid_recommendations` that return top-N ranked movie lists for given users or query movies.

The demo notebook demonstrates typical usage patterns, including fetching similar movies to a given title and generating recommendations for a selected user ID, along with visual inspection of similarity matrices and rating distributions.

3.6 Evaluation and Analysis

The project primarily emphasizes interpretability and educational clarity rather than exhaustive metric-driven evaluation, but it includes several evaluation dimensions. Example evaluation aspects are:

- Coverage: Proportion of users and items for which the system can generate meaningful recommendations under sparsity constraints.
- Diversity: Degree to which recommended lists avoid repetition and span different genres and content clusters.
- Prediction quality: Conceptual discussion of RMSE, MAE, and ranking metrics such as Precision@K, which can be implemented as extensions.

The small size of the dataset enables manual inspection of recommendation lists and similarity structures, which is valuable for understanding the qualitative behavior of each algorithm (e.g., content-based filter bubbles vs. collaborative exploration). The README explicitly proposes adding formal metrics such as RMSE, MAE, and Precision@K as future work, framing the current implementation as a baseline system ready for quantitative enhancement.

3.7 Results and Discussions

Commands to run backend is shown below after executing ,

```
PS D:\movie-recommendation-system> cd backend
PS D:\movie-recommendation-system\backend> uv run recommendation_engine.py
📁 Initializing Movie Recommendation Engine

✅ Loaded 50 movies, 100 users, 1000 ratings
🔧 Initializing recommendation engine...
✅ Content similarity matrix built: (50, 50)
✅ User similarity: (100, 100)
✅ Item similarity: (50, 50)
✅ Recommendation engine ready!

=====
DEMO: Content-Based Recommendations
=====

👤 Based on: The Last Horizon (1997)
   Genres: Fantasy

📁 Similar Movies:
movie_id  title  year  genres  avg_rating  similarity_score
    38  Lost Paradise  2011  Comedy  7.7  0.345046
    7  The Forgotten War  2000  Sci-Fi|Thriller  7.4  0.309812
   44  Midnight Sun  2011  Biography  6.1  0.306713
   27  The Awakening  2023  War|Romance  7.9  0.305105
   10  The Lost Kingdom  1998  Fantasy  6.0  0.294802
```

```
=====
DEMO: Hybrid Recommendations
=====

👤 Recommendations for User ID: 10

📁 Recommended Movies:
movie_id  title  year  genres  avg_rating  predicted_rating  hybr
id_score
    18  Whispers in the Dark  2004  Thriller|Action|Comedy  7.3  4.929811
4.350868
    14  Crimson Dawn  2001  War|Comedy|Crime  9.3  4.923688
4.346582
    7  The Forgotten War  2000  Sci-Fi|Thriller  7.4  4.939831
4.057881
    23  The Revolution  2003  War|Action  8.1  4.931206
4.051844
    39  The Witness  2010  Documentary|Animation  5.0  4.925782
4.048047

✅ Demo complete!
```

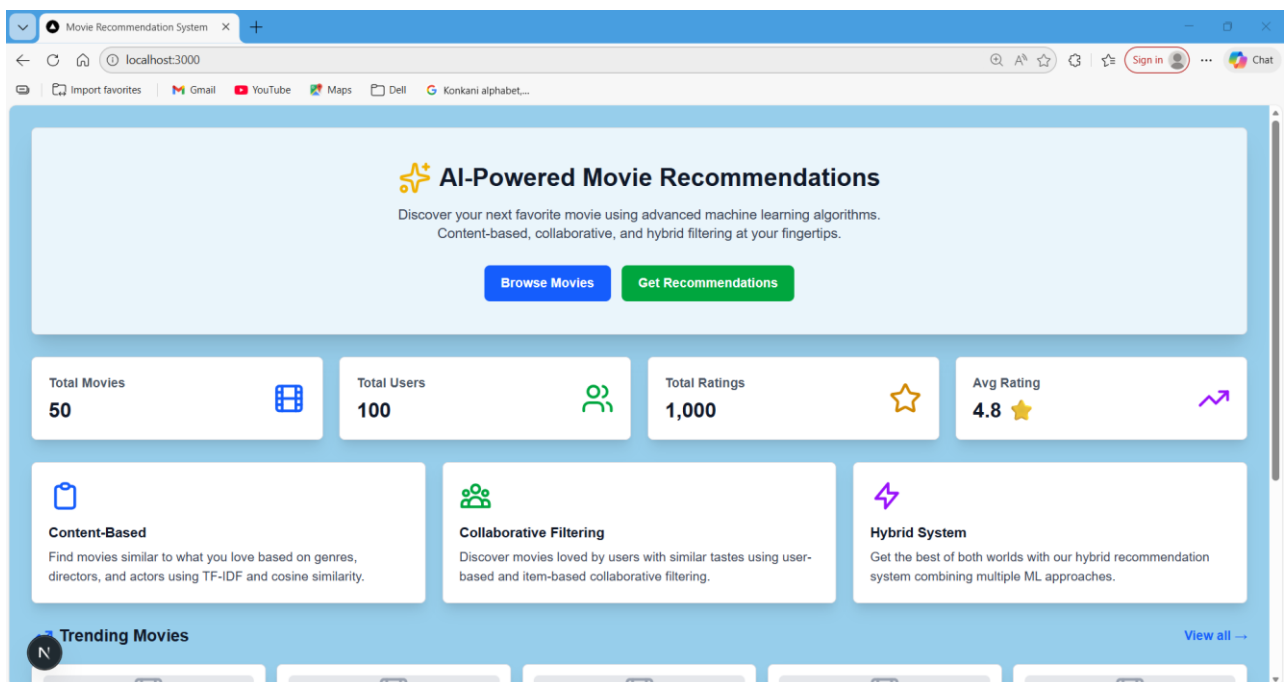
Commands to run frontend is shown below after executing ,

```
PS D:\movie-recommendation-system> cd frontend
PS D:\movie-recommendation-system\frontend> npm run dev

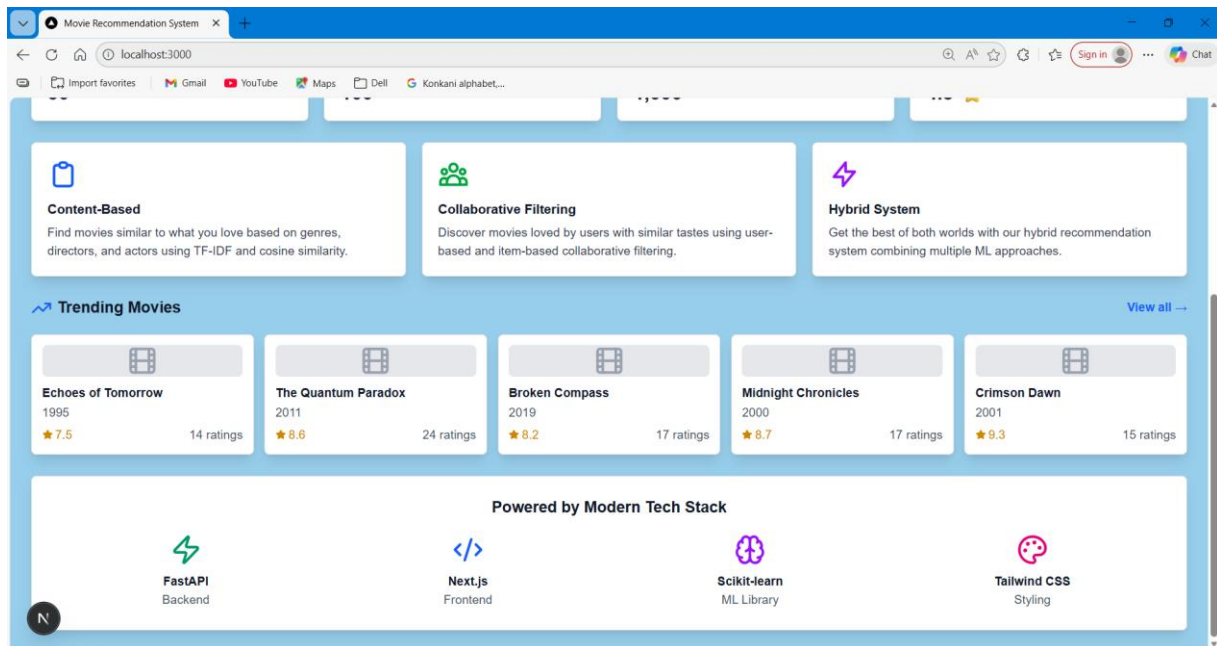
> frontend@0.1.0 dev
> next dev
> next dev

▲ Next.js 16.1.1 (Turbopack)
- Local:      http://localhost:3000
- Network:    http://192.168.1.33:3000
```

After clicking using **ctrl + click** on the link , <http://localhost:3000> we get the web interface page or localhost page of Movie Recommendation System as shown below ,



Contd...



3.8 Strengths, Limitations and Future Work

i) Strengths :

The system offers several pedagogical and practical strengths:

Clear, modular implementation of three major recommendation paradigms (content-based, user-based collaborative, item-based collaborative) plus a hybrid layer.

Realistic data characteristics (sparsity, mixed genres, heterogeneous user preferences) while keeping the dataset small and fully inspectable.

Emphasis on similarity matrices and matrix operations, providing strong foundations for students transitioning to more advanced techniques like matrix factorization.

ii) Limitations :

At the same time, the project intentionally simplifies several aspects:

The dataset scale is limited to 50 movies and 100 users, which does not stress test scalability, distributed computation, or latency issues typical in production recommender systems.

The algorithms are memory-based and rely on TF-IDF and cosine similarity; there is no model-based collaborative filtering (e.g., SVD, ALS) or deep learning-based recommenders (NCF, autoencoders, transformers).

Temporal dynamics, context (time of day, device), and multi-objective optimization (e.g., balancing relevance with novelty or fairness) are not explicitly modeled.

iii) Future Work :

The README and architecture documents propose multiple directions for future work suitable for an M.S. thesis extension:

Integrate matrix factorization techniques such as SVD or ALS to learn latent user–item factors and compare performance with memory-based methods.

Incorporate deep learning models (Neural Collaborative Filtering, autoencoders, sequence models) to capture complex interaction patterns and temporal sequences.

Deploy the recommendation engine as a scalable API using FastAPI, with Redis caching and possibly a microservices design for online inference.

Implement A/B testing and logging to measure click-through rates, watch-time, and engagement, feeding results back into model selection and tuning.

Extend the dataset or switch to real-world data to evaluate robustness, scalability, and generalization under larger catalogs and user bases.

Chapter - 4

CONCLUSION

During my internship at CADDESK Mysuru, I worked on the design and development of a Movie Recommendation System, which helped me gain a strong understanding of how intelligent recommendation engines operate using data-driven approaches. I learned how different recommendation techniques—such as content-based filtering, collaborative filtering, and hybrid recommendation models—analyze user preferences and item characteristics to generate personalized recommendations. The project also enhanced my understanding of system workflows, data preprocessing, feature engineering, and the integration of machine learning models into practical applications.

This internship significantly contributed to my technical and practical skill development. I gained hands-on experience in Python programming, data manipulation using Pandas and NumPy, and implementing similarity measures such as cosine similarity and TF-IDF vectorization. I also developed skills in designing modular system architecture, building reusable components, and integrating the recommendation engine with backend frameworks for real-world deployment. Working on this project improved my ability to translate theoretical concepts into functional systems and strengthened my confidence in developing data-driven applications.

Looking ahead, there is substantial scope to extend this work by enhancing the scalability and intelligence of the recommendation system. Future improvements may include integrating deep learning-based recommendation models, incorporating real-time user feedback, handling large-scale datasets using big data technologies, and improving personalization through context-aware and hybrid learning techniques. The foundation built during this internship has prepared me to contribute effectively to advanced recommender system research and industry-level AI applications in both academic and professional environments.

Chapter - 5

APPLICATIONS

Online Streaming Platforms

Movie recommendation systems are widely used in online streaming services such as Netflix, Amazon Prime Video, and Disney+. These systems analyze user viewing history, ratings, and preferences to suggest personalized movies, thereby enhancing user engagement and retention.

Video-on-Demand (VoD) Services

In video-on-demand platforms, recommendation systems help users discover relevant movies from large content libraries. Personalized suggestions reduce search time and improve overall user satisfaction.

E-Commerce and Digital Marketplaces

Movie recommendation systems are integrated into e-commerce platforms that sell or rent digital movies. By recommending movies based on previous purchases and browsing behavior, these systems increase sales and customer loyalty.

Smart Television and Set-Top Boxes

Modern smart TVs and set-top boxes use recommendation systems to suggest movies and shows based on user viewing patterns, time of viewing, and content popularity, providing a customized entertainment experience.

Content Discovery and Media Libraries

Digital media libraries and movie databases use recommendation systems to help users explore similar movies based on genre, actors, directors, and themes, enabling efficient content discovery.

Personalized Advertising and Marketing

Movie recommendation systems support targeted advertising by suggesting promotional content, trailers, or subscription plans tailored to user preferences, improving marketing effectiveness.

Social Media and Entertainment Apps

Entertainment-focused social media platforms use movie recommendation systems to suggest trending or relevant movies based on user interactions, likes, and shared content.

Educational and Research Platforms

In film studies and media research, recommendation systems assist students and researchers in discovering movies related to specific genres, styles, or historical periods.

Chapter – 6

REFERENCES

- Google , ChatGPT
- D:\movie-recommendation-system